DevOps Fundamentals

Linux, Git, Docker and Jenkins | Laboratory work 1

Student field	Value
Name	Akku
Surname	Zhanabergen
Group	IT2-2309
Student ID	37667
Course	DevOps
Date	17 October 2026

Purpose and verified result

The project follows Linux → Git → GitHub → Jenkins → Build → Test → Docker Build → Docker Run. The application displays the student's information and reads the four STUDENT_* environment variables.

Check	Result
GitHub	https://github.com/akkuzhanabergen/zhanabergen-akku-devops
Git history	8 source commits on main; development merged
Python tests	4 passed
Docker Compose	Application and health endpoint returned HTTP 200
Jenkins	Build #1 finished SUCCESS

Linux environment

The project directory is /root/devops/zhanabergen_akku inside a Linux container. It contains app, scripts, config, logs and backup. The preparation script creates a ten-line log and sets file permissions after cloning because *.log and backup/ are ignored by Git.

Command or check	Observed result
find . -name application.log	./logs/application.log
grep ERROR logs/application.log	Two ERROR lines
grep WARNING logs/application.log	Two WARNING lines
grep -c ERROR logs/application.log	2
tail -n 5 logs/application.log	Displayed the five final messages
ls -l private.txt	-rw------- : only owner reads and writes
ls -l public.txt	-rw-r--r-- : other users may read
Zhanabergen_Akku_system.sh	Printed student, username, hostname, date, OS, disk and memory; found info file

Commands demonstrated

pwd shows the current directory; ls lists files; cd changes directory; mkdir creates a directory; touch creates an empty file; cp copies it; mv renames or moves it; rm removes it; cat prints a file; head and tail show its first and last lines. find searches paths and grep selects matching lines.

Permissions r, w and x mean read, write and execute for owner, group and others. The private file uses 600 and the public file uses 644. ps gives a process snapshot, while top refreshes CPU and memory usage. The running sleep process had PID 1, owner root, 0% CPU and 0% displayed memory usage in top. A PID is the process identifier.

Git and GitHub

Git tracks the source and its changes; GitHub hosts the public repository used by Jenkins. The local repository has meaningful commits. A development branch was created, changed and merged into main. The .gitignore excludes *.log, backup/ and .env, which keeps generated logs, backups and local settings out of commits.

Recent commit	Message
eb6b017	Merge development health check
a9e0def	Test the HTTP health endpoint
a35e1eb	Add five-stage Jenkins pipeline
26f9a34	Containerize student application with Compose
597d967	Add student application and tests
8c7c2b4	Add Linux system and log preparation scripts
7271b03	Add student information and configuration

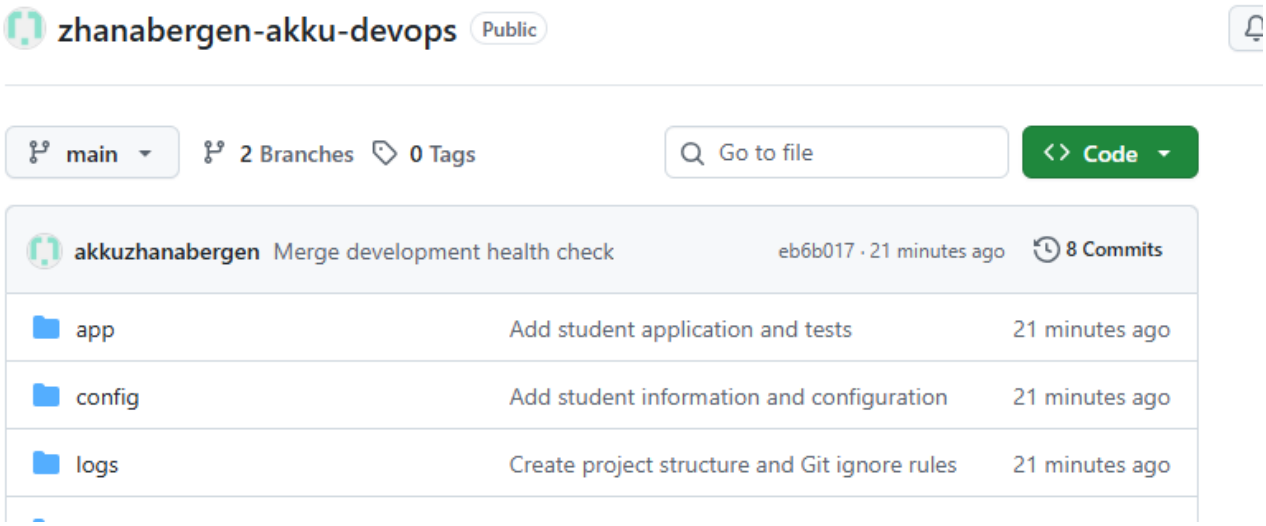


Figure 1. Public GitHub repository with project files and commit history.

git status checks local changes; git log shows commits; git branch shows branches; git remote -v confirms the GitHub remote; git push sends local commits to GitHub. Branches let work be changed separately before it is merged.

Docker

Dockerfile uses FROM, WORKDIR, COPY, RUN, EXPOSE and CMD. The image packages the Python application. A container is a running instance of that image. The application listens on container port 8000, exposed locally on port 8002.

Operation	Verified outcome
docker build	Image zhanabergen-akku-devops built
docker run with -e	Container displayed name, surname, group and ID
docker logs	Startup information printed
docker stop / start / rm	Container lifecycle commands succeeded
docker compose up / down	Service started, returned HTTP 200, then stopped

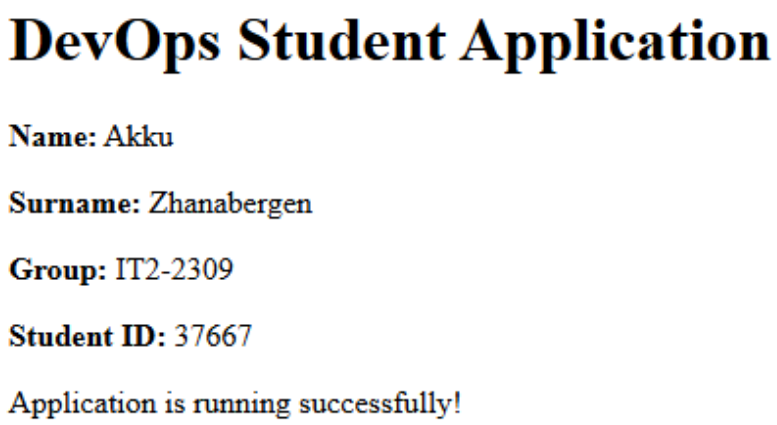


Figure 2. Student application running in the Docker container.

Environment variables provide values when a container starts, so one image can be reused with different student data. Sensitive values should be supplied at runtime rather than embedded in an image. Docker Compose stores the build, environment and port settings in docker-compose.yml.

Jenkins Pipeline

Job Zhanabergen_Akku_DevOps reads the Jenkinsfile from https://github.com/akkuzhanabergen/zhanabergen-akku-devops. Build #1 used commit eb6b017aa9439748dc7cbfe7c8be8ae73eca8f64 and finished successfully.

Stage	Action
Checkout	Fetch GitHub main and print the commit
Build	Compile the Python application
Test	Run four unit tests
Docker Build	Build zhanabergen-akku-devops
Docker Run	Start container, check /health and print logs

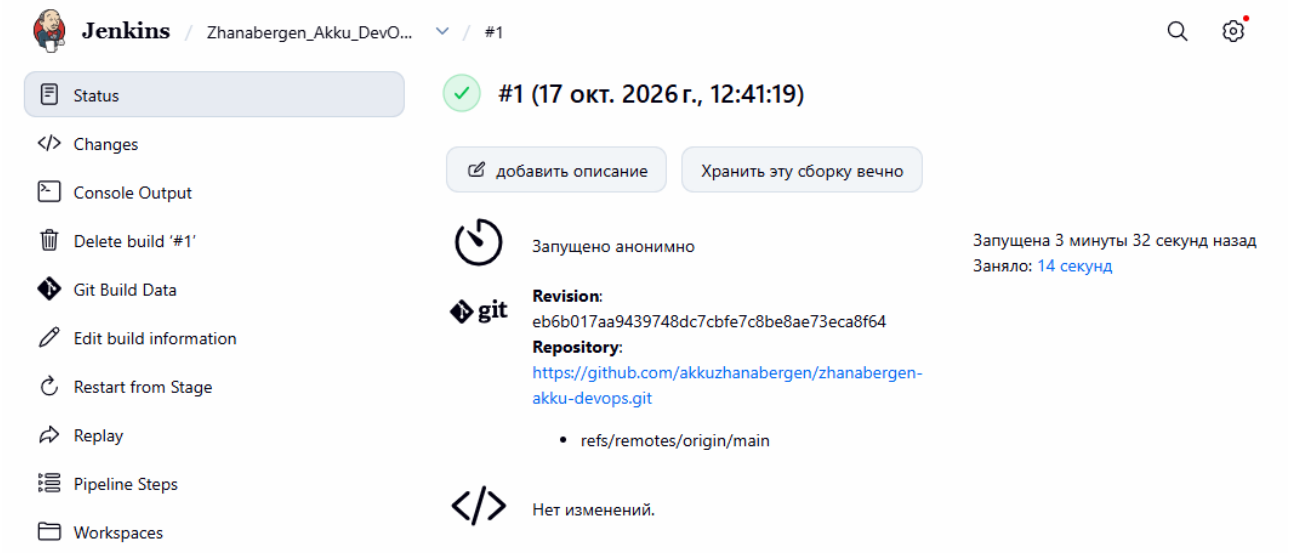


Figure 3. Jenkins Job build #1 with successful status, GitHub URL and Git revision.

Pipeline output

The Console Output displayed Akku Zhanabergen, IT2-2309, Student ID 37667 and Finished: SUCCESS. The Docker container responded with HTTP 200 at /health.

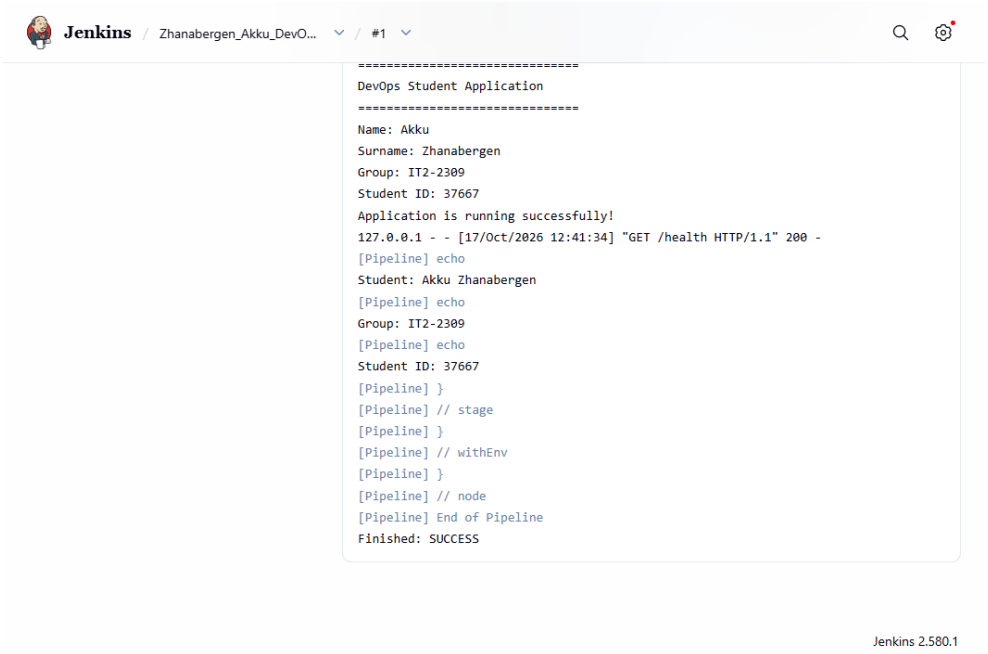


Figure 4. Jenkins Console Output with student data and final status.

Integrated DevOps workflow

Linux supplies the shell, files and permissions. Git records each change. GitHub stores the repository and Jenkins reads its main branch. Jenkins checks out a specific commit, builds and tests the source, builds a Docker image and runs a container. If any stage fails, later stages stop and Jenkins records the error in Console Output.

Conclusion

The project, repository, Linux commands, Docker image and Compose service were verified locally. The Jenkins Pipeline completed successfully using the GitHub commit shown above. The required continuous video must be recorded and submitted separately by the student using their own screen and voice.